

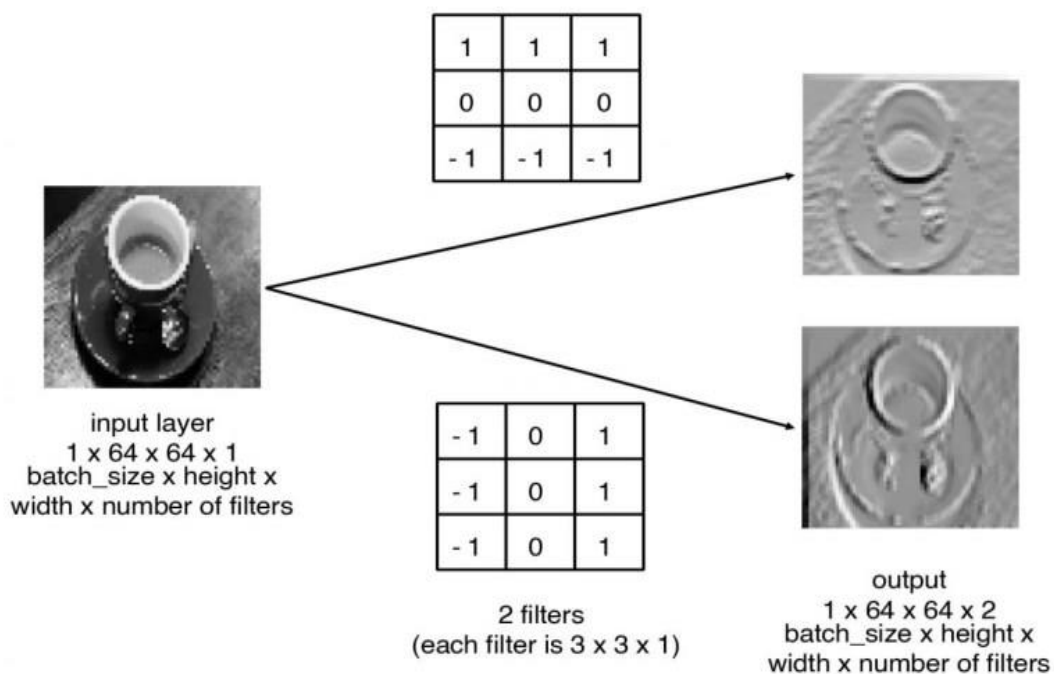
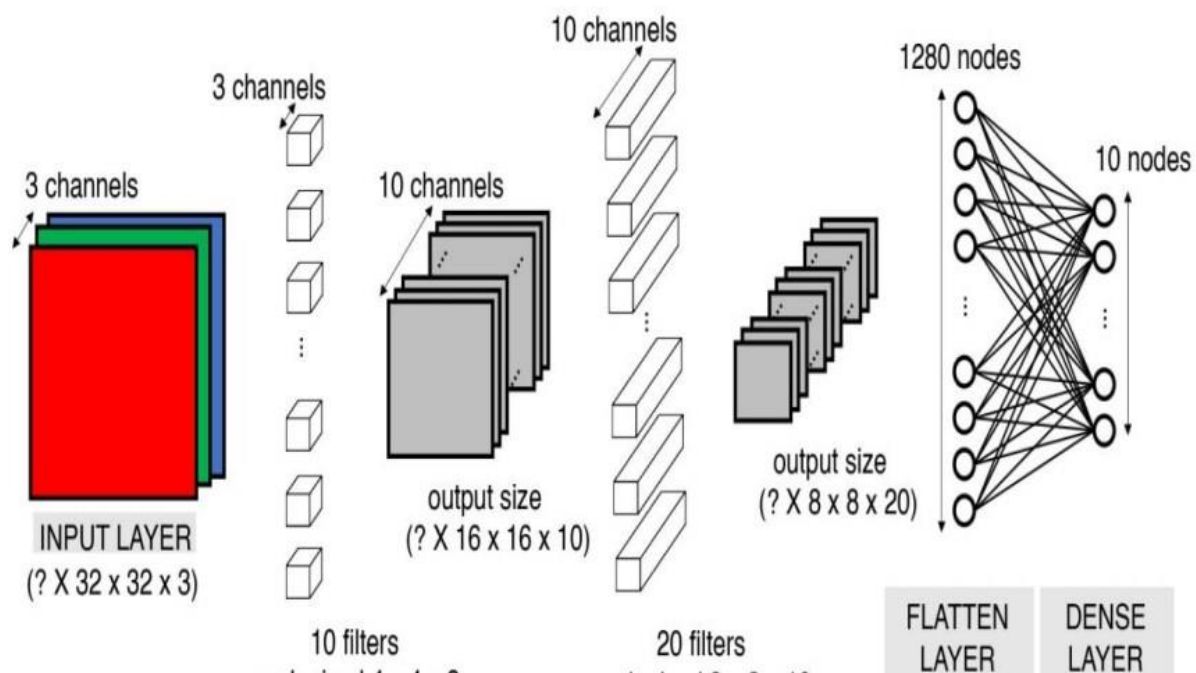


Figure 2-11. Two convolutional filters applied to a grayscale image

3x3 portion  
of an image

|      |      |      |
|------|------|------|
| 0.6  | 0.2  | 0.6  |
| 0.1  | -0.2 | -0.3 |
| -0.5 | -0.1 | -0.3 |

\*

filter

|    |    |    |
|----|----|----|
| 1  | 1  | 1  |
| 0  | 0  | 0  |
| -1 | -1 | -1 |

= 2.3

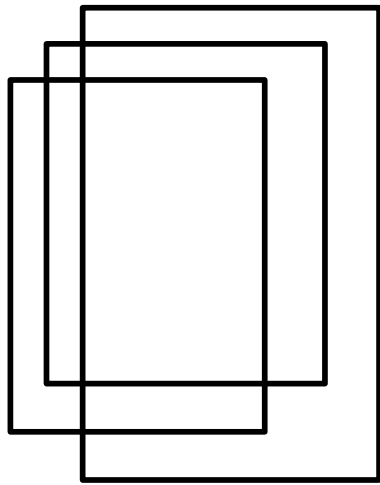
|      |      |      |
|------|------|------|
| -0.6 | -0.2 | -0.6 |
| -0.1 | 0.2  | 0.3  |
| 0.5  | 0.1  | 0.3  |

\*

|    |    |    |
|----|----|----|
| 1  | 1  | 1  |
| 0  | 0  | 0  |
| -1 | -1 | -1 |

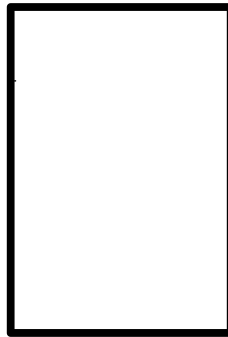
= -2.3

*Figure 2-10. The convolution operation*



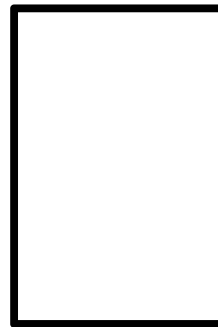
RGB Image 227X227X3

R 227X227



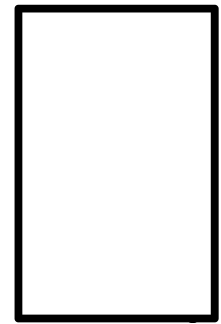
R filter 11X11

G 227X227



G filter 11X11

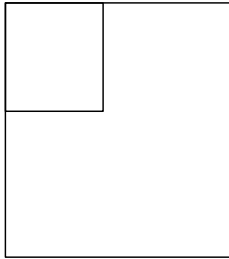
B 227X227



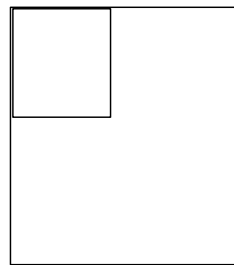
B filter 11x11

Filter 11X11X3

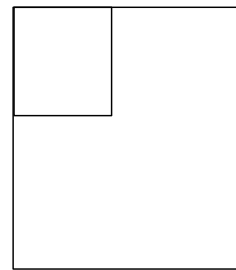
R Filter on R component



G Filter on G component

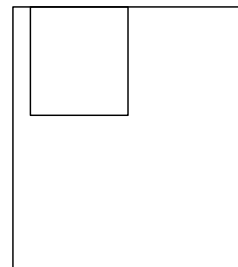
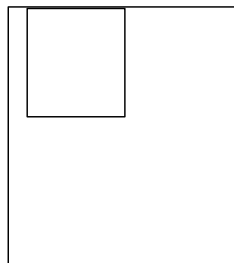
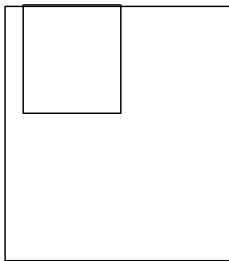


B Filter on B component

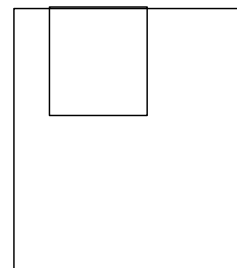
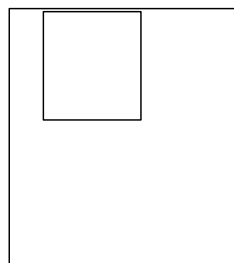
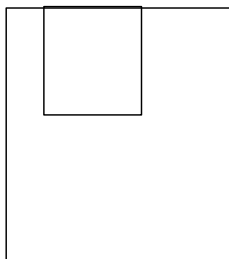


$$\begin{aligned}
 &R1F1+R2F2+R3F3+.....+ \\
 &R121F121+ \\
 &G1F1+G2F2+G3F3+.....+ \\
 &G121F121+ \\
 &B1F1+B2F2+B3F3+.....+ \\
 &B121F121
 \end{aligned}$$

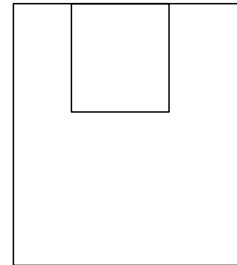
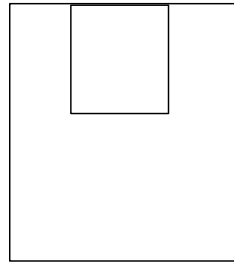
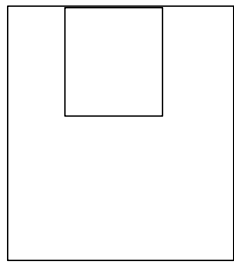
120



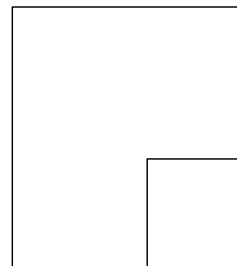
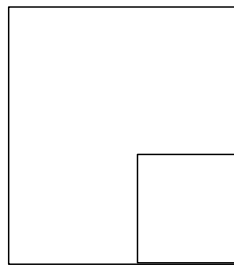
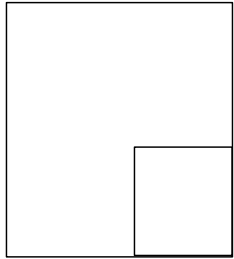
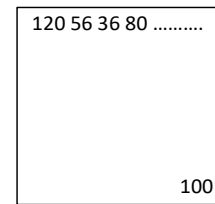
56



36



80



100

Output dimension  
 $= (W-F+2P)/s + 1$   
 $= 55$

Here  $W = 227$ ,  $F = 11$ ,  $P = 0$  and  $S = 4$

If we use 90 filters of size  $11 \times 11 \times 3$  we will produce 90 outputs each having size  $55 \times 55$

Each convolution operation is performed by a neuron. It is connected with  $11 \times 11 \times 3 = 363$  R, G, B values through links having weights equal to the filter values. The initial weight values are set randomly and learned during training through gradient descent approach.

To cover the whole image we need 55 convolution operations horizontally and 55 operations vertically by sliding the filters across the image horizontally and vertically. Thus the output is a single feature map of dimension  $55 \times 55$ .

Each separate filter is supposed to extract a unique kind of feature. That is why all the  $55 \times 55$  image segments are convolved with the same filter having same weights. Thus to produce a feature map using a  $11 \times 11 \times 3$  filter we need  $55 \times 55$  neurons having a single set of weights  $11 \times 11 \times 3$ .

If we use 90 filters each having dimension  $11 \times 11 \times 3$ , we will produce 90 different feature maps each having size  $55 \times 55$ . Total number of neurons will be  $55 \times 55 \times 90$  and the total number of shared weights are  $90 \times 11 \times 11 \times 3$  and 90 biases

In general feature map generated from convolution operations has a size less than the original input. In order to keep the size equal, the input image/feature map is padded with zeros around it. It is called zero padding. Sometimes the outer most rows and columns are copied around more accuracy.

Image =  $W1 \times H1 \times D1$ , Filter =  $F \times F \times D1$ , Number of filters =  $K$ , Stride =  $S$ ,  $P$  = Number of Zero padding

Output =  $W2 \times H2 \times D2$  where  $W2 = (W1 - F + 2P) / S + 1$ ,  $H2 = (H1 - F + 2P) / S + 1$  and  $D2 = K$

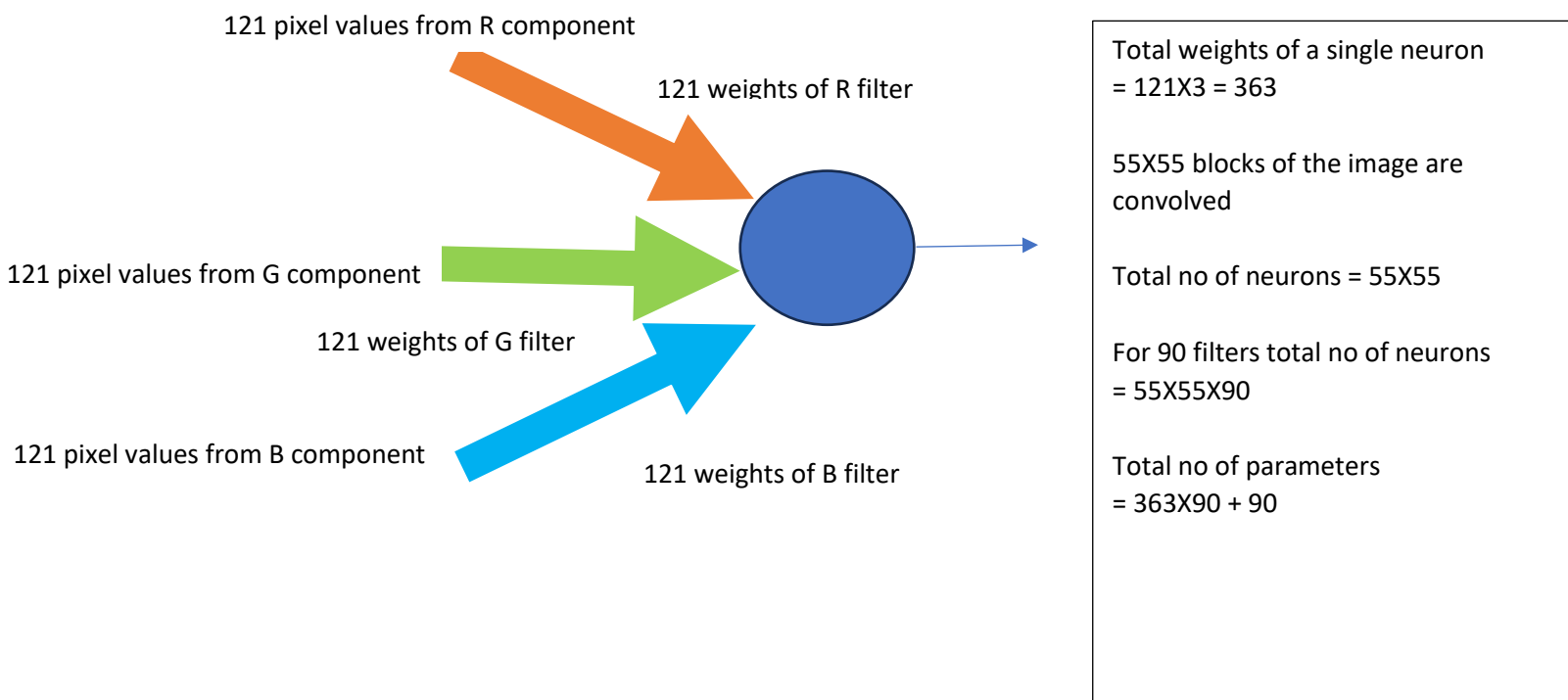
To apply  $K$  filters we need  $F \times F \times D1$  weights per filter, a total  $K \times F \times F \times D1$  weights and  $K$  biases

Convolution operation as matrix multiplication

Image:  $11 \times 11 \times 3$  is the number of pixels convolved with the filter as the filter slides through the image. The image can be divided into  $55 \times 55 = 3,025$  blocks where each block consists of  $11 \times 11 \times 3 = 363$  pixels. So the input image  $227 \times 227 \times 3$  can be represented by  $363 \times 3025$  image matrix where each column represents a convolution block.

Each filter can be represented by a row of  $11 \times 11 \times 3 = 363$  values (weights) and we have 90 such filters. So the filters can be represented by  $90 \times 363$  filter matrix

Image matrix =  $(90 \times 363) \times (363 \times 3025) = 90 \times 3025$



## pooling Layer

|     |     |
|-----|-----|
| 1 8 | 2 6 |
| 2 6 | 0 9 |
| 3 5 | 7 8 |
| 0 1 | 2 0 |

|     |
|-----|
| 8 9 |
| 5 8 |

Max Pooling (2X2 and  $S = 2$ ). Pooling halves the input feature map. The discarded values are marked and all the backward connections through these discarded values are ignored while training through backpropagation is done. There is another variation of pooling which is called average pooling. Sometimes pooling is avoided by applying bigger strides

## Normalization Layer

Normalization in Convolutional Neural Networks (CNNs) is important for several reasons:

### 1. Stability During Training:

Normalization helps in stabilizing the learning process by maintaining a consistent scale of inputs throughout the network, reducing the possibility of fluctuations in parameter updates.

### 2. Improved Convergence:

By ensuring that inputs to each layer have a zero mean and unit variance (or similar targets), normalization can significantly speed up the convergence of the network during training, enabling faster learning.

### 3. Reduced Sensitivity to Initialization:

Normalization can lessen the sensitivity of the network to the initialization of weights. This means models are less reliant on choosing "perfect" initial weights and are more likely to converge to good solutions starting from a broader range of initializations.

### 4. Mitigation of Internal Covariate Shift:

Internal covariate shift refers to changes in the distribution of inputs to a layer as the network learns. Normalization helps in reducing these shifts, maintaining consistent distribution of inputs across layers as the network learns.

### 5. Enhanced Generalization:

Networks with normalization tend to generalize better to new data because they are less prone to overfitting. Normalization acts as a form of regularization.

### 6. Allowance for Higher Learning Rates:

By controlling the scale dynamics within the network, normalization often permits the use of higher learning rates, which can accelerate the training process.

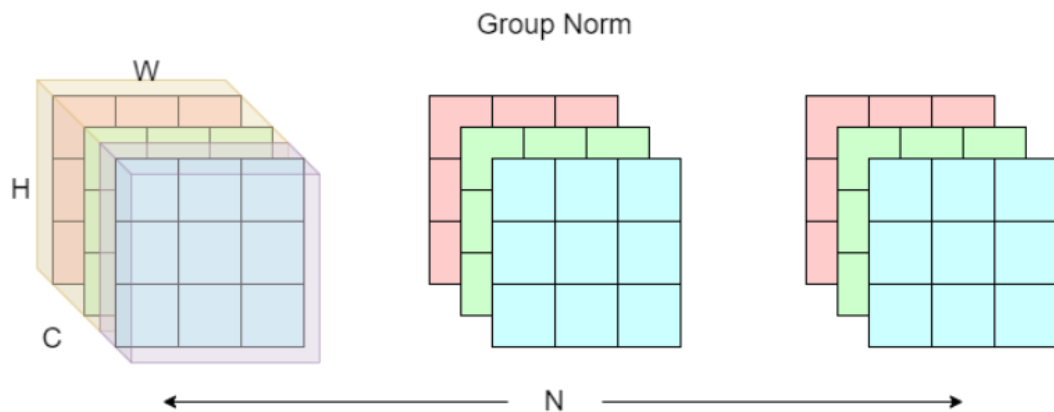
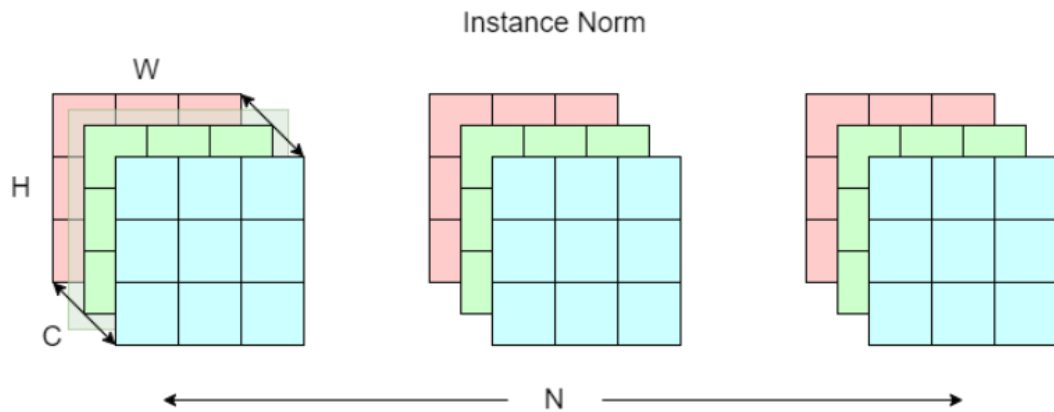
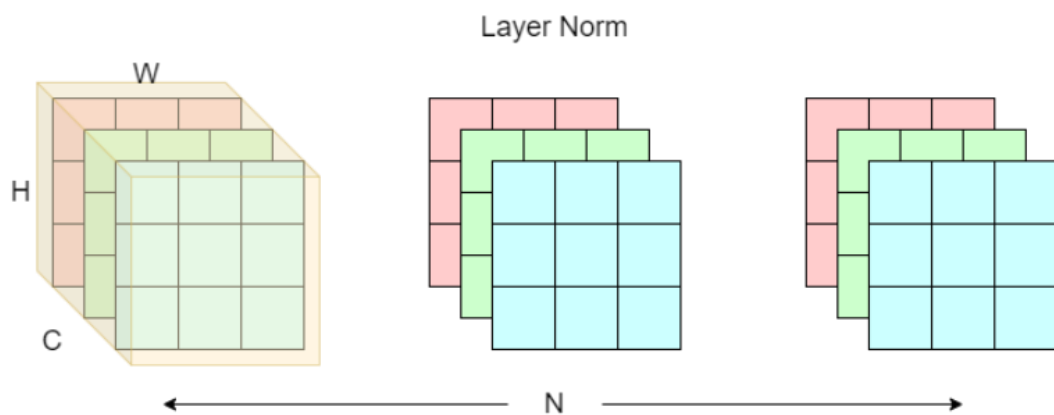
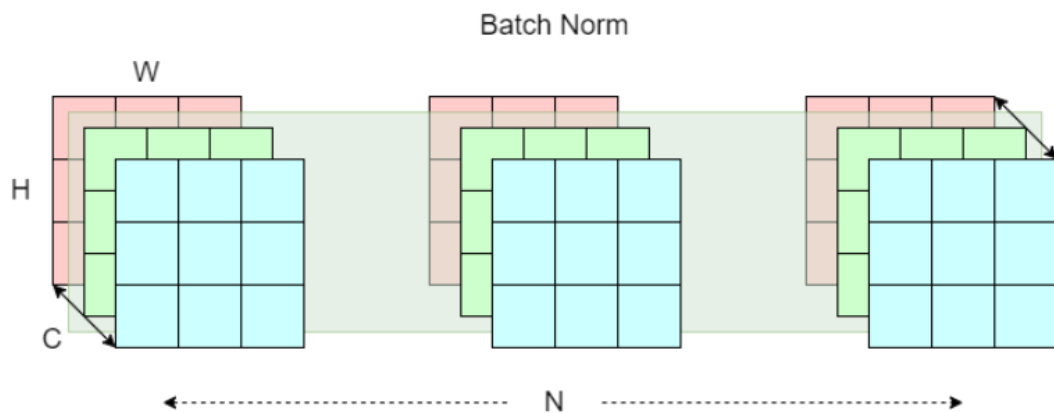
In summary, normalization is a vital component in modern neural network architectures, enabling more efficient and stable training processes, which are crucial for achieving high-performance models.

Normalization has been a standard technique for vision-related tasks for a while, and there are dozens of different strategies out there. It can be overwhelming to try to understand each of them.

Recall that no matter what strategy we pick, the goal of normalization is to “shift” the target samples into a certain distribution. This is usually good for stabilizing training, as normalization standardizes the input.

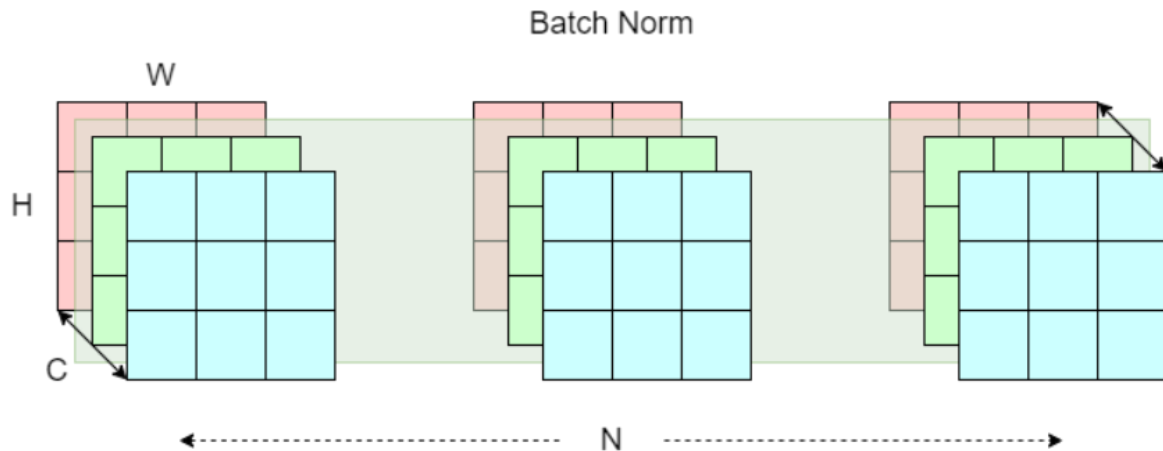
Let’s look at how the top 4 most used normalization strategies work, and why you might choose one over another. Throughout this post will be looking at an example with an input dimension of  $N \times C \times H \times W$ , where  $N$  is the batch size,  $C$  is channels, and  $H \times W$  is the size of an image.

The diagram below illustrates the mechanics behind batch, layer, instance, and group normalization. The shades indicate the scope of each normalization, and the solid lines represent the axis on which the normalizations are applied.



common normalization strategies

## Batch Normalization

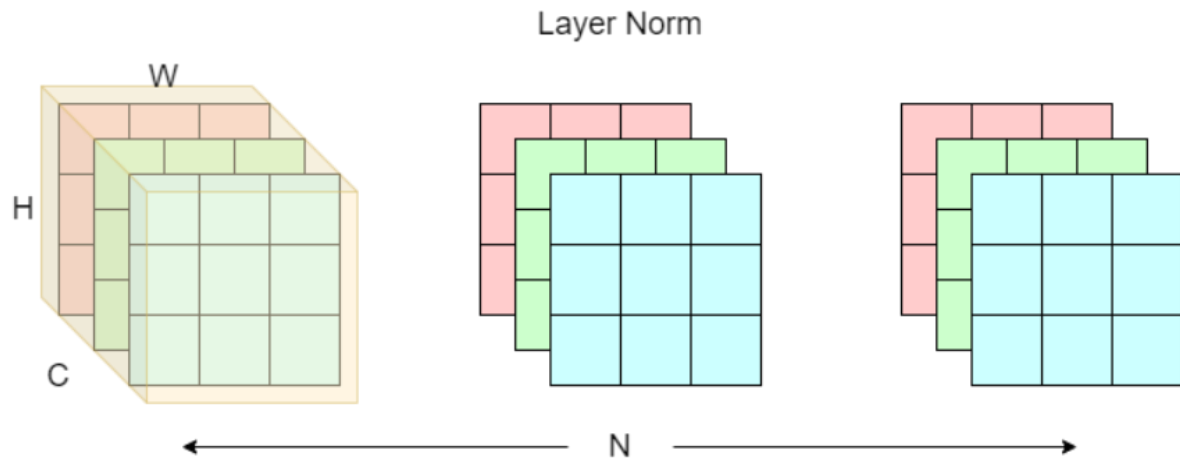


Batch normalization (BN) is the first normalization strategy ever introduced to the world of deep neural networks. Benefits of using batch norm include faster training speed and greater input stability, which significantly reduces the chance of gradient explosion/vanishing.

This technique involves normalizing on the “batch layer” (represented as the green shade)  $N \times H \times W$  across the channel dimension  $C$ .

Quoting from [this article](#), think of the structure of an image as a book. Each book ( $C \times H \times W$ ) has multiple pages ( $C$ ), with each page having height and width  $H \times W$ . Now think of a row of  $N$  books on a bookshelf as a batch of images ( $N \times C \times H \times W$ ). To apply batch norm, we pick the first page of each book and apply normalization. We then repeat this process for the rest of  $C - 1$  pages.

A problem with batch norm is that it yields poor results when there are not enough samples ( $N$ ). It can also be time-consuming due to the way it is implemented. This is not ideal in certain scenarios such as tasks with low batch size, or tasks with variable batch size (online learning).



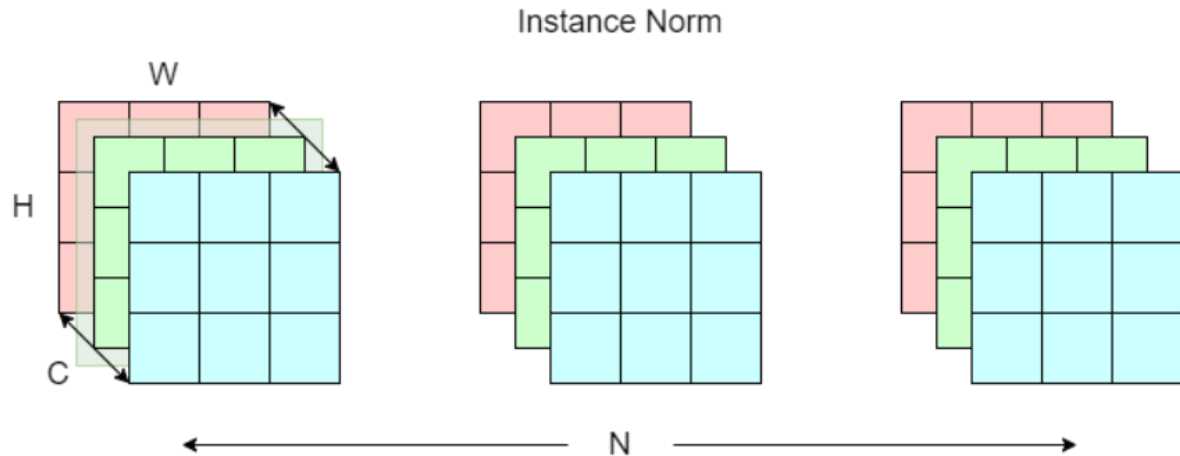
### layer normalization

Layer normalization (LN) fixes the sample size issue that batch norm suffers from. This technique involves normalizing on the “layers” (yellow shade)  $C \times H \times W$ , which is basically a single image sample, across the batch dimension  $N$ . Note that layer norm is not batch dependent.

Continuing our analogy to books, think of the structure of an image as a book. Each book ( $C \times H \times W$ ) has multiple pages ( $C$ ), with each page having height and width  $H \times W$ . Now think of a row of  $N$  books on a bookshelf as a batch of images ( $N \times C \times H \times W$ ). To apply layer norm, we pick the first book ( $C \times H \times W$ ) and apply normalization. We then repeat this process for the rest of  $N - 1$  books.

Layer norm is widely used in sequential modeling (RNN) and the transformer architecture, as NLP tasks often have variable batch sizes (sentence length), it is more intuitive to just normalize on a single word with layer norm.

### Instance Normalization

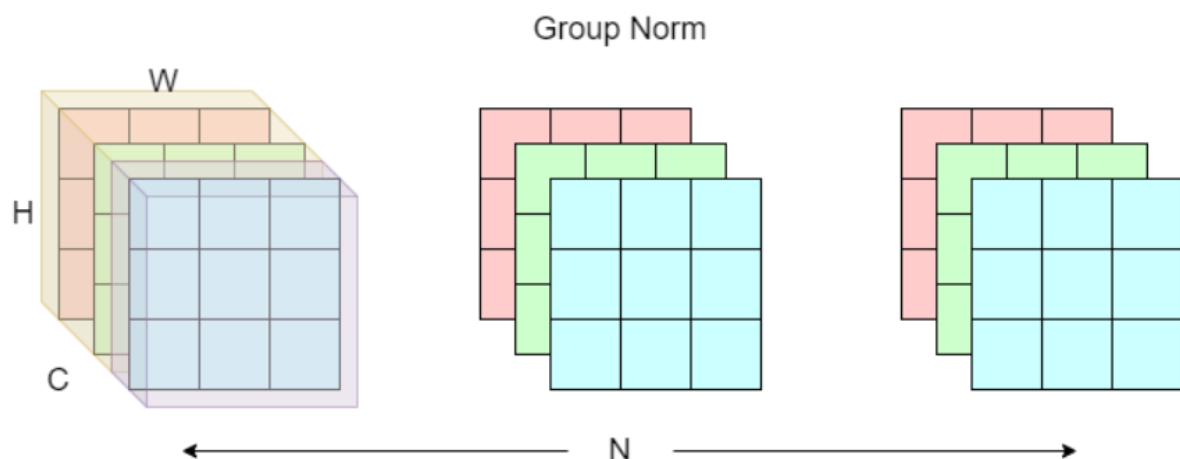


instance normalization

Instance norm (IN), also known as contrast normalization, is first used in the StyleNet paper for transferring image style. This technique involves normalizing on a single “layer” (green shade)  $H \times W$  across both the channel (C) and batch (N) dimensions.

Again, think of the structure of an image as a book. Each book ( $C \times H \times W$ ) has multiple pages (C), with each page having height and width  $H \times W$ . Now think of a row of N books on a bookshelf as a batch of images ( $N \times C \times H \times W$ ). To apply instance norm, we normalize on the first page of the first book ( $H \times W$ ). We then repeat this process for the rest of the  $C - 1$  pages for  $N - 1$  books.

### Group Normalization



group normalization

Group normalization (GN) is a mixture of instance and layer norm. The idea is to split the channel dimensions into “groups” (yellow and purple shades). Normalization is then computed on each group across the batch N dimension just like instance norm.

### Note

For easy comparison, the two groups (yellow and purple shades) given in the diagrams above have different channel counts. This is usually not the case.

Same thing, think of the structure of an image as a book. Each book ( $C \times H \times W$ ) has multiple pages ( $C$ ), with each page having height and width  $H \times W$ . Now think of a row of  $N$  books on a bookshelf as a batch of images ( $N \times C \times H \times W$ ). To apply group norm, we first divide the pages into  $C / G$  groups. We then normalize within every group in the first book. Finally, repeat this process for the rest of  $N - 1$  books.

### Conclusion

Here we summarize the normalization strategies mentioned above.

- Batch norm treat each  $N \times H \times W$  as a unit of normalization.
- Layer norm treat each  $C \times H \times W$  as a unit of normalization.
- Instance norm treat each  $H \times W$  as a unit of normalization.
- Group norm first divides the channel dimension into  $C / G$  groups, then treats each  $(C / G) \times H \times W$  as a unit of normalization.

**VGGNet in detail.** Lets break down the [VGGNet](#) in more detail as a case study. The whole VGGNet is composed of CONV layers that perform  $3 \times 3$  convolutions with stride 1 and pad 1, and of POOL layers that perform  $2 \times 2$  max pooling with stride 2 (and no padding). We can write out the size of the representation at each step of the processing and keep track of both the representation size and the total number of weights:

```
INPUT: [224x224x3]          memory: 224*224*3=150K weights: 0
CONV3-64: [224x224x64]      memory: 224*224*64=3.2M weights: (3*3*3)*64 = 1,728
CONV3-64: [224x224x64]      memory: 224*224*64=3.2M weights: (3*3*64)*64 = 36,864
POOL2: [112x112x64]         memory: 112*112*64=800K weights: 0
CONV3-128: [112x112x128]    memory: 112*112*128= 1.6M weights: (3*3*64)*128 = 73,728
CONV3-128: [112x112x128]    memory: 112*112*128= 1.6M weights: (3*3*128)*128 = 147,456
POOL2: [56x56x128]          memory: 56*56*128=400K weights: 0
CONV3-256: [56x56x256]      memory: 56*56*256=800K weights:
```

$(3*3*128)*256 = 294,912$   
 CONV3-256: [56x56x256] memory: 56\*56\*256=800K weights:  
 $(3*3*256)*256 = 589,824$   
 CONV3-256: [56x56x256] memory: 56\*56\*256=800K weights:  
 $(3*3*256)*256 = 589,824$   
 POOL2: [28x28x256] memory: 28\*28\*256=200K weights: 0  
 CONV3-512: [28x28x512] memory: 28\*28\*512=400K weights:  
 $(3*3*256)*512 = 1,179,648$   
 CONV3-512: [28x28x512] memory: 28\*28\*512=400K weights:  
 $(3*3*512)*512 = 2,359,296$   
 CONV3-512: [28x28x512] memory: 28\*28\*512=400K weights:  
 $(3*3*512)*512 = 2,359,296$   
 POOL2: [14x14x512] memory: 14\*14\*512=100K weights: 0  
 CONV3-512: [14x14x512] memory: 14\*14\*512=100K weights:  
 $(3*3*512)*512 = 2,359,296$   
 CONV3-512: [14x14x512] memory: 14\*14\*512=100K weights:  
 $(3*3*512)*512 = 2,359,296$   
 CONV3-512: [14x14x512] memory: 14\*14\*512=100K weights:  
 $(3*3*512)*512 = 2,359,296$

weights:  $(3*3*512)*512 = 2,359,296$   
POOL2:  $[7*7*512]$  memory:  $7*7*512=25K$  weights: 0 FC:  
 $[1*1*4096]$  memory: 4096 weights:  $7*7*512*4096 =$   
 $102,760,448$   
FC:  $[1*1*4096]$  memory: 4096 weights:  $4096*4096 =$   
 $16,777,216$   
FC:  $[1*1*1000]$  memory: 1000 weights:  $4096*1000 =$   
 $4,096,000$

TOTAL memory:  $24M * 4 \text{ bytes} \approx 93MB$  / image (only  
forward!  $\sim *2$  for bwd)

TOTAL params: 138M parameters